

RÉSUMÉ

R possède des particularités qui peuvent surprendre quand on vient de Python, mais qui rendent le langage particulièrement puissant pour l'analyse de données. Ce poster présente cinq mécanismes qui changent la manière d'écrire du code : des opérateurs lisibles, du dispatch léger, des expressions que l'on peut capturer, un système objet simple et des arguments évalués seulement quand ils sont utilisés.

IDÉE CENTRALE

L'« avancé » n'a pas à être compliqué.

Ces outils sont petits, composables, et souvent déjà présents dans les packages que l'on utilise tous les jours.

S3 métaprogrammation lazy evaluation

LES 5 MÉCANISMES

01 Créer ses opérateurs

Nommer une intention directement dans la syntaxe.

```
`%||` <- function(x, y) {  
  if (is.null(x)) y else x  
}  
  
NULL %||% 10
```

USAGE	COMMENT ÇA MARCHE
Rendre le code plus proche du vocabulaire métier.	Un opérateur infix est une fonction dont le nom est entouré par des `%`.

DEPUIS PYTHON Souvent remplacé par une fonction nommée ; ici, l'intention reste dans l'appel.

02 Surcharger des opérateurs

Adapter une syntaxe connue à vos propres objets.

```
`+.vec2` <- function(a, b) {  
  vec2(a$x + b$x, a$y + b$y)  
}  
  
p1 + p2
```

USAGE	COMMENT ÇA MARCHE
Garder une API naturelle pour des objets spécialisés.	R cherche une méthode S3 nommée comme l'opérateur et la classe.

DEPUIS PYTHON Même idée que `\_\_add\_\_`, mais portée par le système de méthodes de R.

03 Capturer une expression

Traiter du code comme une donnée manipulable.

```
label <- function(expr) {  
  deparse(substitute(expr))  
}  
  
label(log(x + 1))
```

USAGE	COMMENT ÇA MARCHE
Créer des messages, formules ou pipelines plus lisibles.	`substitute()` récupère l'expression avant son évaluation.

DEPUIS PYTHON Proche d'une manipulation d'AST, avec beaucoup moins de cérémonie.

04 Utiliser S3 simplement

Faire de l'orienté objet sans cérémonie.

```
describe <- function(x) {  
  UseMethod('describe')  
}  
  
describe.lm <- function(x) {  
  'un modèle linéaire'  
}
```

USAGE	COMMENT ÇA MARCHE
Écrire une fonction générique puis spécialiser son comportement.	Le dispatch appelle `describe.<classe>()` selon la classe de l'objet.

DEPUIS PYTHON Pas besoin de définir une hiérarchie complète pour un comportement polymorphe.

05 Profiter de la lazy evaluation

Définir des valeurs par défaut qui dépendent des autres arguments.

```
scale2 <- function(  
  x,  
  mu = mean(x),  
  sigma = sd(x)  
) {  
  (x - mu) / sigma  
}
```

USAGE	COMMENT ÇA MARCHE
Offrir des defaults intelligents sans demander plus à l'utilisateur.	Les arguments sont des promesses : ils sont évalués seulement si nécessaire.

DEPUIS PYTHON Évite souvent les sentinelles du type `None` puis les tests manuels.

À RETENIR

R est plus qu'une syntaxe pour manipuler des tableaux.

Ces mécanismes expliquent pourquoi beaucoup de packages R peuvent proposer une interface concise, expressive et agréable à lire. Les comprendre permet d'écrire du code plus clair, pas forcément plus complexe.

PUBLIC	OBJECTIF
Utilisateurs R débutants ou confirmés.	Démystifier les mécanismes dits « avancés ».